TITLE : Develop a Java application demonstrating the use of ArrayList, Vector, and StringBuffer for dynamic data storage, management, and string manipulation.

Problem Statement

Develop a Java application demonstrating the use of ArrayList, Vector, and StringBuffer for dynamic data storage, management, and string manipulation.

Learning Objectives

To implement dynamic data storage and string manipulation using ArrayList, Vector, and StringBuffer.

Theory

ArrayList and Vector are dynamic array implementations provided by the Java Collections Framework. ArrayList offers faster performance in single-threaded applications, whereas Vector provides synchronized operations for thread safety. StringBuffer is a mutable sequence of characters that allows efficient string modification without creating new objects. These classes are commonly used in applications requiring dynamic memory allocation and efficient string processing.

Conclusion

This Java application successfully demonstrates dynamic data storage and thread-safe operations using ArrayList and Vector, alongside efficient string manipulation via StringBuffer. By contrasting ArrayList's high-performance dynamic resizing with Vector's synchronized thread safety, the program illustrates how to choose appropriate collection frameworks based on application concurrency requirements. Additionally, implementing StringBuffer highlights the performance advantages of using mutable character sequences to eliminate the memory overhead associated with immutable String object creation. Combined, these utility classes showcase best practices for dynamic memory allocation and efficient data management in Java.

Exercise

1. Create a **To-Do List application** using ArrayList to store tasks and StringBuffer to display tasks.

Code:

```java
import java.util.ArrayList;
import java.util.Scanner;

public class TodoListApp {
    Run main | Debug main
    public static void main(String[] args) {
        ArrayList<String> tasks = new ArrayList<>();
        Scanner sc = new Scanner(System.in);

        while (true) {
            System.out.println("\n1. Add Task  2. View Tasks  3. Exit");
            System.out.print("Choice: ");
            int choice = sc.nextInt();
            sc.nextLine();

            if (choice == 1) {
                System.out.print("Enter task: ");
                tasks.add(sc.nextLine());
            } else if (choice == 2) {
                StringBuffer sb = new StringBuffer("--- TO-DO LIST ---\n");
                for (int i = 0; i < tasks.size(); i++) {
                    sb.append(i + 1).append(". ").append(tasks.get(i)).append("\n");
                }
                System.out.print(sb);
            } else if (choice == 3) {
                break;
            }
        }
        sc.close();
    }
}
```

Output:

```
Listening on 55778
User program running

1. Add Task  2. View Tasks  3. Exit
Choice:
→ 1

Enter task:
→ Take the trash out


1. Add Task  2. View Tasks  3. Exit
Choice:
→ 1

Enter task:
→ Buy apples


1. Add Task  2. View Tasks  3. Exit
Choice:
→ 2

--- TO-DO LIST ---
1. Take the trash out
2. Buy apples

1. Add Task  2. View Tasks  3. Exit
Choice:
→ 3

User program finished
```

2. Create a **Student Course Registration System** using **ArrayList** to store the list of courses registered by a student and **StringBuffer** to generate and display the registered course list. The application should allow users to add, remove, and view registered courses.

Code:

```java
import java.util.ArrayList;
import java.util.Scanner;
public class StudentCourseRegistration {
    Run main | Debug main
    public static void main(String[] args) {
        ArrayList<String> courses = new ArrayList<>();
        Scanner sc = new Scanner(System.in);
        while (true) {
            System.out.println("\n1. Add Course | 2. Remove Course | 3. View Courses | 4. Exit");
            System.out.print("Select option: ");
            int choice = sc.nextInt();
            sc.nextLine();
            if (choice == 1) {
                System.out.print("Enter course name: ");
                String course = sc.nextLine();
                courses.add(course);
                System.out.println("Course added.");
            } else if (choice == 2) {
                System.out.print("Enter course name to remove: ");
                String course = sc.nextLine();
                if (courses.remove(course)) {
                    System.out.println("Course removed.");
                } else {
                    System.out.println("Course not found.");
                }
            } else if (choice == 3) {
                if (courses.isEmpty()) {
                    System.out.println("No registered courses.");
                } else {
                    StringBuffer sb = new StringBuffer("\n=== Registered Courses ===\n");
                    for (int i = 0; i < courses.size(); i++) {
                        sb.append(i + 1).append(". ").append(courses.get(i)).append("\n");
                    }
                    sb.append("Total Courses: ").append(courses.size());
                    System.out.println(sb);
                }
            } else if (choice == 4) {
                System.out.println("Exiting system.");
                break;
            } else {
                System.out.println("Invalid option.");
            }
        }
        sc.close();
    }
}
```

Output:

```
Listening on 52990
User program running

1. Add Course | 2. Remove Course | 3. View Courses | 4. Exit
Select option:
→ 1

Enter course name:
→ mathematics

Course added.

1. Add Course | 2. Remove Course | 3. View Courses | 4. Exit
Select option:
→ 1

Enter course name:
→ physics

Course added.

1. Add Course | 2. Remove Course | 3. View Courses | 4. Exit
Select option:
→ 2

Enter course name to remove:
→ mathematics

Course removed.

1. Add Course | 2. Remove Course | 3. View Courses | 4. Exit
Select option:
→ 3
```

```
=== Registered Courses ===
1. physics
Total Courses: 1

1. Add Course | 2. Remove Course | 3. View Courses | 4. Exit
Select option:
→ 4

User program finished
Exiting system.
```